20/8/26

Practical 6

Implement error detection and correction using Hamming Code Concept

Aim:

To write a program to implement error detection and correction using hamming code concept. Make a test run to input data stream and verify error correction feature.

Error Correction at Data Link Layer:

Hamming code is a set of error-correction codes that can be used to detect and correct the errors that can occur when the data is transmitted from the sender to the receiver. It is a technique developed by R.W. Hamming for error correction.

Create sender program with below features:

1. Input to sender file should be a text of any length. Program should convert the text to binary.
2. Apply hamming code concept on the binary data and add redundant bits to it.
3. Save this output in a file called channel

Create a receiver program with below features:

1. Receiver program should read the input from channel file.
2. Apply hamming code on the binary data to check for errors.
3. If there is an error, display the position of the error.
4. Else remove the redundant bits and convert the binary data to ascii and display the output.

Sender.py

Program:

```
def text_to_binary(text):
    binary = ""
    for char in text:
        binary += format(ord(char), '08b')
    return binary

def calculate_parity_bits(data):
    m = len(data)
    r = 0
    while (2**r) < (m + r + 1):
        r += 1
    return r

def hamming_encode(data):
    m = len(data)
    r = calculate_parity_bits(data)
    total_bits = m + r
    hamming = ['0'] * (total_bits + 1)

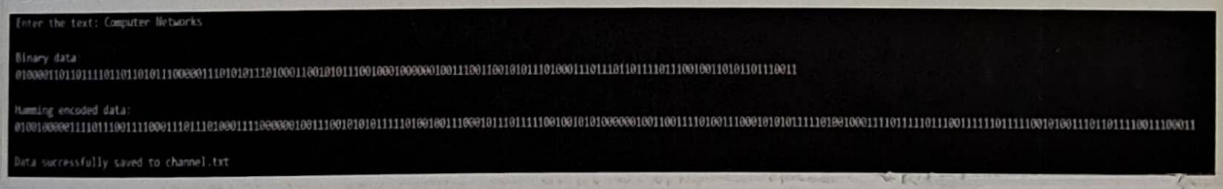
    j = 0
    for i in range(1, total_bits + 1):
        if (i & (i-1)) != 0:
            hamming[i] = data[j]
            j += 1
    for p in range(r):
        position = 2**p
        parity = 0
        for i in range(1, total_bits + 1):
            if i & position:
                parity ^= int(hamming[i])
        hamming[position] = str(parity)
```

```
    return ''.join(hamming[1:])
text = input("Enter the text: ")
binary_data = text_to_binary(text)
print("\nBinary data:")
print(binary_data)
encoded_data = ""
for i in range(0, len(binary_data), 8):
    block = binary_data[i:i + 8]
    encoded_data += hamming_encode(block)
print("\nHamming encoded data:")
print(encoded_data)
with open("channel.txt", "w") as file:
    file.write(encoded_data)
print("\nData successfully saved to channel.txt")
```

Input:

Enter the text: Computer Networks

Output:



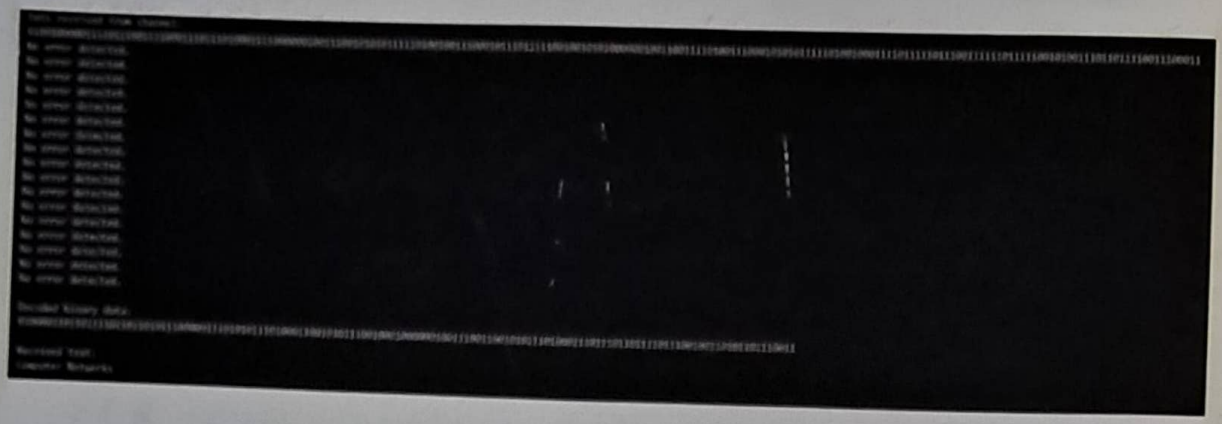
receiver.py

Program

```
def hamming_decode(code):
    n = len(code)
    hamming = ['0'] + list(code)
    r = 0
    while (2**r) <= n:
        r += 1
    error_position = 0
    for p in range(r):
        position = 2**p
        parity = 0
        for i in range(1, n+1):
            if i & position:
                parity ^= int(hamming[i])
        if parity != 0:
            error_position += position
    if error_position != 0:
        print("Error detected at position:",
              error_position)
        if error_position <= n:
            if hamming[error_position] == '0':
                hamming[error_position] = '1'
            else:
                hamming[error_position] = '0'
        print("Error corrected.")
    else:
        print("No error detected.")
```

```
    data = ""
    for i in range(1, n+1):
        if (i & (i-1)) != 0:
            data += hamming[i]
    return data
def binary_to_text(binary):
    text = ""
    for i in range(0, len(binary), 8):
        byte = binary[i:i+8]
        if len(byte) == 8:
            text += chr(int(byte, 2))
    return text
with open("channel.txt", "r") as file:
    channel_data = file.read().strip()
print("Data received from channel:")
print(channel_data)
decoded_binary = ""
for i in range(0, len(channel_data), 12):
    block = channel_data[i:i+12]
    if len(block) == 12:
        decoded_binary += hamming_decode(block)
print("\nDecoded binary data:")
Print(decoded_binary)
received_text = binary_to_text(decoded_binary)
print("\nReceived text:")
print(received_text)
```

Output:



Result:

Thus the error detection and correction using hamming code concept has been implemented.